

```
In [ ]: import marimo as mo
```

VQ-VAE Tokenizer Evaluation (v4 — spherical multi-head)

Spherical multi-feature VQ-VAE の学習後評価ノートブック。Z-matrix 版から変わった点：

- **descriptor**: pocket centroid 起点の spherical (r , θ , $\sin \phi$, $\cos \phi$) + element / charge / hybrid / aromatic / ring / numH (ligand)、AA (protein)。
- **decoder**: multi-head (連続 coord head + 各 categorical head)。
- 評価指標も per-head に再編：coord は Cartesian 空間 MSE、categorical は classification accuracy。3D RMSD (per-atom / Kabsch / joint Kabsch) は旧 baseline と同じ計算式で出すので apples-to-apples 比較できる。

```
In [ ]: import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch

project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

from src.config import CrossDockedConfig, VQVAETrainingConfig
from src.data.descriptors import ComplexDescriptorDataModule
from src.model.vqvae_module import VQVAEModule
from src.tokenizers.descriptor_schema import (
    LIGAND_ELEMENT_VOCAB,
    LIGAND_LAYOUT,
    PROTEIN_AA_VOCAB,
    PROTEIN_LAYOUT,
    fields_by_name,
)

plt.rcParams["figure.dpi"] = 120
plt.rcParams.update({
    "font.size": 18,
    "axes.titlesize": 22,
    "axes.labelsize": 18,
    "xtick.labelsize": 16,
    "ytick.labelsize": 16,
    "legend.fontsize": 16,
    "figure.titlesize": 26,
})
```

1. Load checkpoint and data

```
In [ ]: import os

# Override via VQVAE_CKPT env var when exporting per-run PDFs.
CKPT_PATH = os.environ.get(
    "VQVAE_CKPT",
    str(project_root / "checkpoints" / "v4_latest.ckpt"),
)
print(f>Loading: {CKPT_PATH}")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
module = VQVAEModule.load_from_checkpoint(str(CKPT_PATH), map_location=device)
module.eval()
module.to(device)

protein_vqvae = module.protein_vqvae
ligand_vqvae = module.ligand_vqvae
print(f>Device: {device}")
```

Loading: /home/5/uq02055/bs/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/3dvcbp0h/checkpoints/vqvae-epoch=99-val/ligand_coord=0.1501.ckpt

/gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python3.12/site-packages/torch/nn/modules/transformer.py:392: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
warnings.warn(
Device: cuda

```
In [ ]: import os as _os
from pathlib import Path as _Path

config = VQVAETrainingConfig()
data_config = CrossDockedConfig(data_dir=project_root / "data")
dm = ComplexDescriptorDataModule(config, data_config)
cache_override = _os.environ.get("VQVAE_CACHE_DIR", "data/descriptor_cache_v")
cache_path = _Path(cache_override)
if not cache_path.is_absolute():
    cache_path = project_root / cache_path
dm.cache_dir = cache_path
dm.setup()

norm_stats = dm.norm_stats

# The full test split is ~250k complexes; cap at MAX_TEST_COMPLEXES so
# MSE / accuracy / t-SNE statistics stay tractable.
MAX_TEST_COMPLEXES = 2000

protein_test_pockets = []
ligand_test_molecules = []
prot_mean_np = norm_stats["protein_mean"].numpy()
prot_std_np = norm_stats["protein_std"].numpy()
lig_mean_np = norm_stats["ligand_mean"].numpy()
lig_std_np = norm_stats["ligand_std"].numpy()
```

```

for shard_idx, local_indices in dm._test_plan:
    if len(protein_test_pockets) >= MAX_TEST_COMPLEXES:
        break
    shard_path = dm._shard_dir / f"shard_{shard_idx:04d}.pt"
    shard_data = torch.load(shard_path, weights_only=False)
    for local_idx in local_indices:
        if len(protein_test_pockets) >= MAX_TEST_COMPLEXES:
            break
        cplx = shard_data[local_idx]
        protein_test_pockets.append(
            torch.from_numpy(
                (cplx["protein"] - prot_mean_np) / prot_std_np,
            ).float().to(device),
        )
        ligand_test_molecules.append(
            torch.from_numpy(
                (cplx["ligand"] - lig_mean_np) / lig_std_np,
            ).float().to(device),
        )
    del shard_data

protein_test_flat = torch.cat(protein_test_pockets)
ligand_test_flat = torch.cat(ligand_test_molecules)

print(
    f"Protein test: {len(protein_test_pockets)} pockets, "
    f"{protein_test_flat.shape[0]} residues total"
)
print(
    f"Ligand test: {len(ligand_test_molecules)} molecules, "
    f"{ligand_test_flat.shape[0]} atoms total"
)

```

types-file path: using legacy random 80/10/10 split

Protein test: 2000 pockets, 54357 residues total

Ligand test: 2000 molecules, 48118 atoms total

2. Per-head reconstruction quality

Multi-head decoder の各 head ごとに metric を出す。

- **coord (continuous)**: Cartesian 空間で MSE / RMSE。spherical → xyz は pocket canonical frame 上の値で計算するので、復元品質を絶対距離で見られる。
- **categorical heads**: argmax 予測の accuracy。confusion matrix も出す。

```

In [ ]: @torch.no_grad()
def run_vqvae_per_complex(model, sequences, layout):
    """Encode each sequence, decode multi-head, return per-token outputs.

    Returns a dict with stacked tensors:
    - indices: codebook idx per token (N_total,)
    - z: pre-quant latent per token (N_total, latent_dim)
    - heads: dict mapping head name → (N_total, dim) prediction
    """

```

```

- input: original normalized descriptor (N_total, D)
"""
all_indices, all_z = [], []
head_outputs: dict[str, list[torch.Tensor]] = {
    name: [] for name, _kind, _dim in model.recon_heads
}
all_input = []

for seq in sequences:
    if seq.shape[0] == 0:
        continue
    # encoder embed (mirrors TransformerVQVAE.forward up to z)
    x = seq.unsqueeze(0)
    h_in = model._embed_descriptor(x)
    h = model.input_proj(model.input_norm(h_in)) + model.pos_encoding[:
    h = model.transformer_encoder(h)
    z = model.latent_norm(model.latent_proj(h)).squeeze(0)

    quantized, indices, _, _ = model.codebook(z)
    q_seq = model.latent_unproj(quantized).unsqueeze(0)
    dec_in = q_seq + model.pos_encoding[: seq.shape[0]]
    dec_out = model.transformer_decoder(dec_in)
    trunk = model.decoder_trunk(dec_out).squeeze(0)
    for name, _kind, _dim in model.recon_heads:
        head_outputs[name].append(model.recon_head_modules[name](trunk))

    all_indices.append(indices)
    all_z.append(z)
    all_input.append(seq)

return {
    "indices": torch.cat(all_indices),
    "z": torch.cat(all_z),
    "input": torch.cat(all_input),
    "heads": {n: torch.cat(o) for n, o in head_outputs.items()},
    "layout": layout,
}

prot_out = run_vqvae_per_complex(protein_vqvae, protein_test_pockets, PROTEI
lig_out = run_vqvae_per_complex(ligand_vqvae, ligand_test_molecules, LIGAND
print("Inference done.")
print(f"Protein residues: {prot_out['indices'].shape[0]}")
print(f"Ligand atoms:      {lig_out['indices'].shape[0]}")

```

Inference done.

Protein residues: 54357

Ligand atoms: 48118

```

In [ ]: def per_head_metrics(out, vqvae, mean_t, std_t, name):
    """Compute per-head metrics for one VQ-VAE branch."""
    layout = out["layout"]
    f = fields_by_name(layout)
    results = {}

    # --- coord head: Cartesian MSE/RMSE in canonical frame ---
    coord_field = f["coord"]

```

```

coord_norm = out["input"][:, coord_field.start : coord_field.end]
coord_pred = out["heads"]["coord"]
# denormalize and convert spherical → Cartesian
m = mean_t[coord_field.start : coord_field.end].cpu()
s = std_t[coord_field.start : coord_field.end].cpu()
coord_target_norm = coord_norm.cpu()
coord_pred_cpu = coord_pred.cpu()
coord_target_denorm = coord_target_norm * s + m
coord_pred_denorm = coord_pred_cpu * s + m
n_atoms_per_token = coord_field.length // 4
ct = coord_target_denorm.view(-1, n_atoms_per_token, 4)
cp = coord_pred_denorm.view(-1, n_atoms_per_token, 4)
# spherical → cartesian
def sph2cart(t):
    r, theta, sphi, cphi = t[..., 0], t[..., 1], t[..., 2], t[..., 3]
    norm = (sphi * sphi + cphi * cphi).clamp_min(1e-12).sqrt()
    sphi = sphi / norm
    cphi = cphi / norm
    x = r * torch.sin(theta) * cphi
    y = r * torch.sin(theta) * sphi
    z = r * torch.cos(theta)
    return torch.stack([x, y, z], dim=-1) # noqa: PD013
xyz_t = sph2cart(ct)
xyz_p = sph2cart(cp)
# per-token sum of squared distance over the (1 or 3) atoms in the slot
diff_sq = (xyz_t - xyz_p).pow(2).sum(dim=-1) # (N, atoms)
coord_mse_per_atom = diff_sq.mean().item()
coord_rmse_per_atom = float(np.sqrt(coord_mse_per_atom))
results["coord_mse"] = coord_mse_per_atom
results["coord_rmse"] = coord_rmse_per_atom

# --- categorical heads: classification accuracy ---
cat_metrics: dict[str, dict] = {}
for head_name, kind, _dim in vqvae.recon_heads:
    if kind == "continuous":
        continue
    spec = f[head_name]
    target = out["input"][:, spec.start].long().cpu().numpy()
    logits = out["heads"][head_name].cpu().numpy()
    pred = logits.argmax(axis=-1)
    acc = float((pred == target).mean())
    cat_metrics[head_name] = {
        "accuracy": acc,
        "target": target,
        "pred": pred,
    }
results["categorical"] = cat_metrics

print(f"=== {name} VQ-VAE ===")
print(f"  coord per-atom MSE   : {coord_mse_per_atom:.4f} Å²")
print(f"  coord per-atom RMSE  : {coord_rmse_per_atom:.4f} Å")
for head_name, hm in cat_metrics.items():
    print(f"  {head_name:<10s} acc      : {hm['accuracy']:.4f}")
print()
return results

```

```
In [ ]: prot_metrics = per_head_metrics(
        prot_out, protein_vqvae,
        norm_stats["protein_mean"], norm_stats["protein_std"], "Protein",
    )
    lig_metrics = per_head_metrics(
        lig_out, ligand_vqvae,
        norm_stats["ligand_mean"], norm_stats["ligand_std"], "Ligand",
    )
```

=== Protein VQ-VAE ===

```
coord per-atom MSE : 0.8731 Å²
coord per-atom RMSE : 0.9344 Å
aa          acc      : 0.9740
```

=== Ligand VQ-VAE ===

```
coord per-atom MSE : 0.1644 Å²
coord per-atom RMSE : 0.4054 Å
element  acc      : 0.9984
charge   acc      : 1.0000
hybrid   acc      : 0.9885
aromatic acc      : 0.9994
ring     acc      : 0.9765
numH     acc      : 0.9768
```

2.1 Per-head accuracy summary

全 categorical head の accuracy を 1 つの bar chart にまとめる。各 head が天井 (1.0) にどれだけ近いかが一目で分かる。

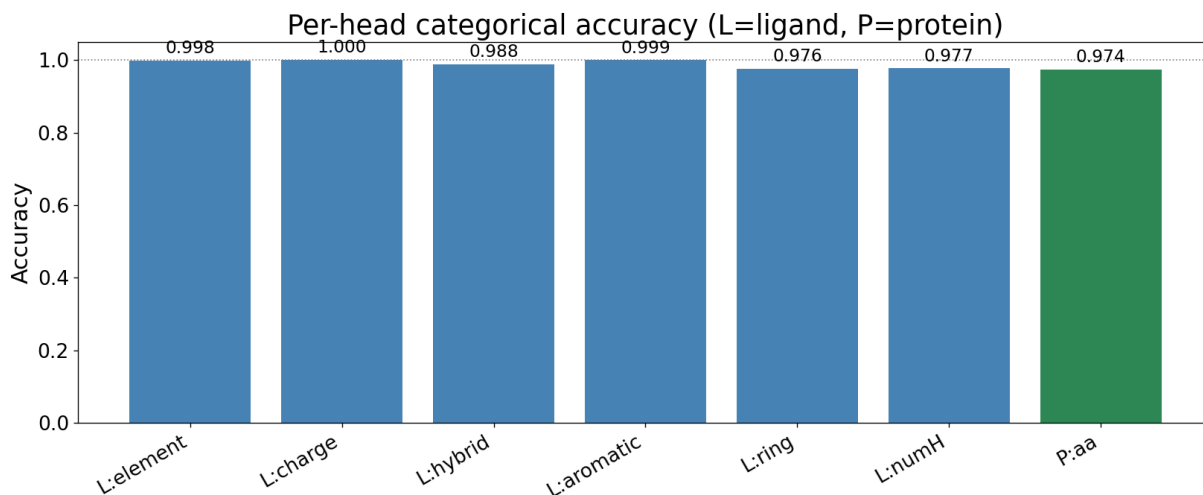
```
In [ ]: lig_cats = lig_metrics["categorical"]
        prot_cats = prot_metrics["categorical"]
        labels = (
            [f"L:{n}" for n in lig_cats]
            + [f"P:{n}" for n in prot_cats]
        )
        accs = (
            [lig_cats[n]["accuracy"] for n in lig_cats]
            + [prot_cats[n]["accuracy"] for n in prot_cats]
        )
        colors = ["steelblue"] * len(lig_cats) + ["seagreen"] * len(prot_cats)

        _fig, _ax = plt.subplots(figsize=(14, 6))
        _bars = _ax.bar(range(len(labels)), accs, color=colors)
        _ax.set_xticks(range(len(labels)))
        _ax.set_xticklabels(labels, rotation=30, ha="right")
        _ax.set_ylabel("Accuracy")
        _ax.set_ylim(0.0, 1.05)
        _ax.axhline(1.0, color="grey", linestyle=":", linewidth=1)
        _ax.set_title("Per-head categorical accuracy (L=ligand, P=protein)")
        for _b, _a in zip(_bars, accs, strict=True):
            _ax.text(
                _b.get_x() + _b.get_width() / 2,
                _a + 0.01,
```

```

        f"{_a:.3f}",
        ha="center",
        va="bottom",
        fontsize=14,
    )
    _fig.tight_layout()
    _fig

```



2.2 Categorical confusion matrices

各 categorical head の confusion matrix。対角線が支配的であれば codebook がその属性を分離できている。小さい vocab (charge / hybrid / aromatic / ring / numH) は格子状に並べる。

```

In [ ]: LIGAND_CHARGE_LABELS = ("-2", "-1", "0", "+1", "+2")
        LIGAND_HYBRID_LABELS = ("SP", "SP2", "SP3", "AROM", "OTHER")
        LIGAND_AROMATIC_LABELS = ("non-arom", "arom")
        LIGAND_RING_LABELS = ("3", "4", "5", "6+", "none")
        LIGAND_NUMH_LABELS = ("0", "1", "2", "3", "4")

        SMALL_VOCAB = 5
        HIGH_CONTRAST = 0.5

        def plot_confusion(target, pred, vocab, ax, title, *, tick_fontsize=14):
            n = len(vocab)
            cm = np.zeros((n, n), dtype=np.int64)
            for t, p in zip(target, pred, strict=False):
                if 0 <= t < n and 0 <= p < n:
                    cm[t, p] += 1
            cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True).clip(min=1)
            ax.imshow(cm_norm, cmap="Blues", vmin=0, vmax=1)
            ax.set_xticks(range(n))
            ax.set_yticks(range(n))
            rotation = 90 if n > SMALL_VOCAB else 0
            ax.set_xticklabels(vocab, rotation=rotation, fontsize=tick_fontsize)
            ax.set_yticklabels(vocab, fontsize=tick_fontsize)
            ax.set_xlabel("Predicted")
            ax.set_ylabel("True")
            ax.set_title(title)

```

```

if n <= SMALL_VOCAB:
    for i in range(n):
        for j in range(n):
            val = cm_norm[i, j]
            color = "white" if val > HIGH_CONTRAST else "black"
            ax.text(
                j,
                i,
                f"{val:.2f}",
                ha="center",
                va="center",
                color=color,
                fontsize=tick_fontsize,
            )

# Large-vocab heads on their own row.
_fig1, _axes1 = plt.subplots(1, 2, figsize=(22, 10))
plot_confusion(
    lig_metrics["categorical"]["element"]["target"],
    lig_metrics["categorical"]["element"]["pred"],
    LIGAND_ELEMENT_VOCAB,
    _axes1[0],
    f"Ligand element (acc={lig_metrics['categorical']['element']['accuracy']}
)
plot_confusion(
    prot_metrics["categorical"]["aa"]["target"],
    prot_metrics["categorical"]["aa"]["pred"],
    PROTEIN_AA_VOCAB,
    _axes1[1],
    f"Protein AA (acc={prot_metrics['categorical']['aa']['accuracy']:.3f})",
)
_fig1.tight_layout()

# Small-vocab ligand heads on a 2x3 grid (5 used, 1 spare).
small_heads = [
    ("charge", LIGAND_CHARGE_LABELS),
    ("hybrid", LIGAND_HYBRID_LABELS),
    ("aromatic", LIGAND_AROMATIC_LABELS),
    ("ring", LIGAND_RING_LABELS),
    ("numH", LIGAND_NUMH_LABELS),
]
_fig2, _axes2 = plt.subplots(2, 3, figsize=(22, 14))
_flat = _axes2.ravel()
for _ax, (_head, _labels) in zip(_flat, small_heads, strict=False):
    _m = lig_metrics["categorical"][_head]
    plot_confusion(
        _m["target"],
        _m["pred"],
        _labels,
        _ax,
        f"Ligand {_head} (acc={_m['accuracy']:.3f})",
    )
_flat[-1].axis("off")
_fig2.tight_layout()
[_fig1, _fig2]

```


3. Codebook utilization & perplexity

```
In [ ]: def codebook_stats(indices, codebook_size, name):
    idx_np = indices.cpu().numpy()
    unique = np.unique(idx_np)
    utilization = len(unique) / codebook_size
    counts = np.bincount(idx_np, minlength=codebook_size).astype(float)
    probs = counts / counts.sum()
    probs_nonzero = probs[probs > 0]
    entropy = -np.sum(probs_nonzero * np.log(probs_nonzero))
    perplexity = np.exp(entropy)

    print(f"=== {name} Codebook ===")
    print(f"  Codebook size:           {codebook_size}")
    print(f"  Active codes:              {len(unique)}")
    print(f"  Utilization:               {utilization:.4f}")
    print(f"  Perplexity:                {perplexity:.1f} / {codebook_size}")
    print(f"  Normalized perplexity:     {perplexity / codebook_size:.4f}")
    print(f"  Dead codes:                {codebook_size - len(unique)}")
    print()
    return counts

prot_counts = codebook_stats(
    prot_out["indices"], config.protein.codebook_size, "Protein",
)
lig_counts = codebook_stats(
    lig_out["indices"], config.ligand.codebook_size, "Ligand",
)

=== Protein Codebook ===
  Codebook size:           4096
  Active codes:            7956
  Utilization:             1.9424
  Perplexity:              6253.4 / 4096
  Normalized perplexity:   1.5267
  Dead codes:              -3860

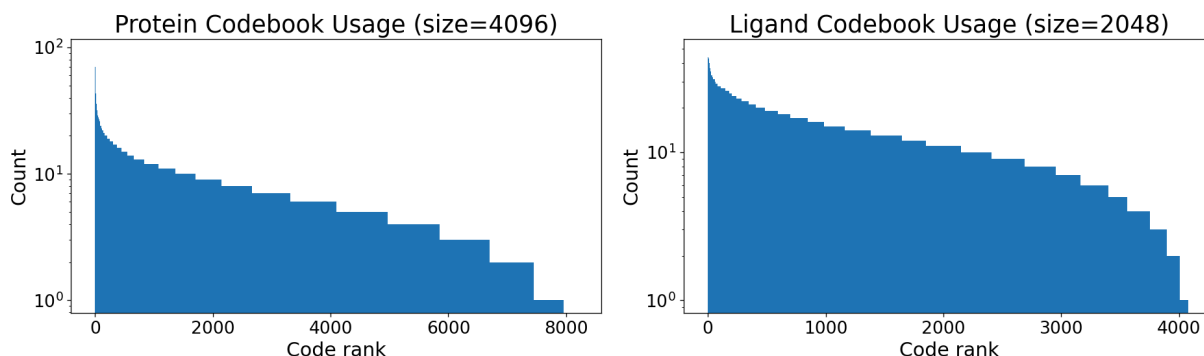
=== Ligand Codebook ===
  Codebook size:           2048
  Active codes:            4078
  Utilization:             1.9912
  Perplexity:              3480.0 / 2048
  Normalized perplexity:   1.6992
  Dead codes:              -2030

In [ ]: _fig, _axes = plt.subplots(1, 2, figsize=(16, 5))
        _axes[0].bar(range(len(prot_counts)), np.sort(prot_counts[::-1]), width=1.0)
        _axes[0].set_xlabel("Code rank")
        _axes[0].set_ylabel("Count")
        _axes[0].set_title(f"Protein Codebook Usage (size={config.protein.codebook_s
        _axes[0].set_yscale("log")
        _axes[1].bar(range(len(lig_counts)), np.sort(lig_counts[::-1]), width=1.0)
        _axes[1].set_xlabel("Code rank")
```

```

_axes[1].set_ylabel("Count")
_axes[1].set_title(f"Ligand Codebook Usage (size={config.ligand.codebook_size})")
_axes[1].set_yscale("log")
_fig.tight_layout()
_fig

```



4. 3D Reconstruction RMSD (Å)

記述子空間ではなく実際の3次元座標に復元した上での RMSD を評価する。Z-matrix baseline と完全に同じ指標で出すので直接比較できる。

- **per-atom**: pocket canonical frame を共有しているので生の RMSD。
- **Kabsch-aligned**: 内部形状 (conformer shape) の精度。
- **Joint Kabsch**: protein backbone + ligand を 1 rigid body として 合わせたときの全原子 RMSD と、その内訳。

```

In [ ]: import pyarrow.parquet as pq

from src.config import PocketExtractionConfig
from src.tokenizers.ligand import LigandDescriptor, parse_sdf
from src.tokenizers.protein import (
    BackboneSphericalDescriptor,
    _compute_canonical_frame,
    extract_pocket,
)

def kabsch_align(p, q):
    p_c = p - p.mean(axis=0)
    q_c = q - q.mean(axis=0)
    h = q_c.T @ p_c
    u, _, vt = np.linalg.svd(h)
    d = np.sign(np.linalg.det(vt.T @ u.T))
    rot = vt.T @ np.diag([1.0, 1.0, d]) @ u.T
    q_aligned = q_c @ rot.T
    rmsd = float(np.sqrt(np.mean(np.sum((p_c - q_aligned) ** 2, axis=-1))))
    return p_c, q_aligned, rmsd

def kabsch_rmsd(p, q):
    return kabsch_align(p, q)[2]

def reconstruct_descriptor(coord_norm, full_dim, coord_field):

```

```

"""Build a (N, full_dim) descriptor populated only at the coord slot."""
n = coord_norm.shape[0]
desc = np.zeros((n, full_dim), dtype=np.float32)
desc[:, coord_field.start : coord_field.start + coord_field.length] = (
    coord_norm.cpu().numpy().astype(np.float32)
)
return desc

pocket_config = PocketExtractionConfig()
protein_desc_calc = BackboneSphericalDescriptor()
ligand_desc_calc = LigandDescriptor()
hub_cache_dir = project_root / "data" / "hub_cache"
manifest_df = pq.read_table(
    hub_cache_dir / "repo" / "manifest.parquet",
).to_pandas()
test_df = manifest_df[
    (manifest_df["source_type"] == "cdonly")
    & (manifest_df["cdonly_fold0"] == "test")
].reset_index(drop=True)
receptor_dir = hub_cache_dir / "receptors"
ligand_dir = hub_cache_dir / "ligands"
entries = [
    (f"{row.complex_dir}/{row.receptor_pdb}", f"{row.pair_idx:07d}.sdf.gz")
    for row in test_df.itertuples(index=False)
]

prot_mean_t = norm_stats["protein_mean"].to(device)
prot_std_t = norm_stats["protein_std"].to(device)
lig_mean_t = norm_stats["ligand_mean"].to(device)
lig_std_t = norm_stats["ligand_std"].to(device)
prot_coord_field = fields_by_name(PROTEIN_LAYOUT)["coord"]
lig_coord_field = fields_by_name(LIGAND_LAYOUT)["coord"]
PROT_DIM = PROTEIN_LAYOUT[-1].end
LIG_DIM = LIGAND_LAYOUT[-1].end

N_SAMPLES = 2000
rng = np.random.default_rng(42)
sample_indices = rng.choice(
    len(entries), size=min(N_SAMPLES * 5, len(entries)), replace=False,
)

prot_rmsd_list, prot_rmsd_aligned_list = [], []
lig_rmsd_list, lig_rmsd_aligned_list = [], []
joint_rmsd_list, prot_in_joint_list, lig_in_joint_list = [], [], []
joint_per_atom_list = []
n_done = 0
for idx in sample_indices:
    if n_done >= N_SAMPLES:
        break
    rec_rel, lig_rel = entries[idx]
    rec_path = receptor_dir / rec_rel
    lig_path = ligand_dir / lig_rel
    if not rec_path.exists() or not lig_path.exists():
        continue
    try:
        molecules = parse_sdf(lig_path)

```

```

if not molecules:
    continue
mol = molecules[0]
heavy = np.array(
    [(a[1], a[2], a[3]) for a in mol["atoms"] if a[0] != "H"],
    dtype=np.float32,
)
pocket_result = extract_pocket(rec_path, heavy, pocket_config)
if pocket_result is None:
    continue
backbone_orig, pocket_seq, residue_ids = pocket_result
ca = backbone_orig[:, 1].astype(np.float64)
centroid, rotation = _compute_canonical_frame(ca)
pocket_frame = (centroid, rotation)

prot_desc, prot_meta = protein_desc_calc.compute(
    backbone_orig, residue_ids,
    pocket_frame=pocket_frame,
    residue_names_one_letter=list(pocket_seq),
)
prot_t = torch.from_numpy(prot_desc).to(device)
prot_norm = (prot_t - prot_mean_t) / prot_std_t
with torch.no_grad():
    pi = protein_vqvae.encode(prot_norm)
    outs = protein_vqvae.decode_to_outputs(pi)
prot_coord_denorm = (
    outs["coord"]
    * prot_std_t[prot_coord_field.start : prot_coord_field.end]
    + prot_mean_t[prot_coord_field.start : prot_coord_field.end]
)
prot_recon_desc = reconstruct_descriptor(
    prot_coord_denorm, PROT_DIM, prot_coord_field,
)
backbone_recon = BackboneSphericalDescriptor.descriptor_to_backbone(
    prot_recon_desc, prot_meta,
)

lig_desc, _elements, lig_meta = ligand_desc_calc.compute(
    mol["atoms"], mol["bonds"], pocket_frame=pocket_frame,
)
if len(lig_desc) == 0:
    continue
lig_t = torch.from_numpy(lig_desc).to(device)
lig_norm = (lig_t - lig_mean_t) / lig_std_t
with torch.no_grad():
    li = ligand_vqvae.encode(lig_norm)
    louts = ligand_vqvae.decode_to_outputs(li)
lig_coord_denorm = (
    louts["coord"]
    * lig_std_t[lig_coord_field.start : lig_coord_field.end]
    + lig_mean_t[lig_coord_field.start : lig_coord_field.end]
)
lig_recon_desc = reconstruct_descriptor(
    lig_coord_denorm, LIG_DIM, lig_coord_field,
)
lig_coords_recon = LigandDescriptor.descriptor_to_coords(

```

```

        lig_recon_desc, lig_meta, pocket_frame=pocket_frame,
    )

    heavy_to_orig = lig_meta["heavy_to_orig"]
    lig_coords_orig = np.array(
        [(mol["atoms"][i][1], mol["atoms"][i][2], mol["atoms"][i][3])
         for i in heavy_to_orig], dtype=np.float64,
    )

    prot_per_atom_rmsd = float(np.sqrt(
        np.mean(np.sum((backbone_orig - backbone_recon) ** 2, axis=-1))
    ))
    prot_flat_orig = backbone_orig.reshape(-1, 3).astype(np.float64)
    prot_flat_recon = backbone_recon.reshape(-1, 3).astype(np.float64)
    prot_kabsch = kabsch_rmsd(prot_flat_orig, prot_flat_recon)

    lig_per_atom_rmsd = float(np.sqrt(
        np.mean(np.sum((lig_coords_orig - lig_coords_recon) ** 2, axis=-1))
    ))
    lig_kabsch = kabsch_rmsd(lig_coords_orig, lig_coords_recon.astype(np.float64))

    joint_orig = np.vstack([prot_flat_orig, lig_coords_orig])
    joint_recon = np.vstack(
        [prot_flat_recon, lig_coords_recon.astype(np.float64)],
    )
    n_prot = len(prot_flat_orig)
    # Per-atom whole-complex RMSD: no alignment, atoms are already in
    # the same global frame because both branches share the pocket
    # canonical frame.
    joint_per_atom = float(np.sqrt(
        np.mean(np.sum((joint_orig - joint_recon) ** 2, axis=-1))
    ))
    joint_orig_c, joint_recon_aligned, joint_rmsd = kabsch_align(
        joint_orig, joint_recon,
    )
    prot_diff_joint = joint_orig_c[:n_prot] - joint_recon_aligned[:n_prot]
    lig_diff_joint = joint_orig_c[n_prot:] - joint_recon_aligned[n_prot:]
    prot_in_joint = float(np.sqrt(
        np.mean(np.sum(prot_diff_joint ** 2, axis=-1))
    ))
    lig_in_joint = float(np.sqrt(
        np.mean(np.sum(lig_diff_joint ** 2, axis=-1))
    ))

    prot_rmsd_list.append(prot_per_atom_rmsd)
    prot_rmsd_aligned_list.append(prot_kabsch)
    lig_rmsd_list.append(lig_per_atom_rmsd)
    lig_rmsd_aligned_list.append(lig_kabsch)
    joint_rmsd_list.append(joint_rmsd)
    joint_per_atom_list.append(joint_per_atom)
    prot_in_joint_list.append(prot_in_joint)
    lig_in_joint_list.append(lig_in_joint)
    n_done += 1
except Exception: # noqa: BLE001, S112
    continue

```

```

prot_rmsd_arr = np.array(prot_rmsd_list)
prot_rmsd_aligned_arr = np.array(prot_rmsd_aligned_list)
lig_rmsd_arr = np.array(lig_rmsd_list)
lig_rmsd_aligned_arr = np.array(lig_rmsd_aligned_list)
joint_rmsd_arr = np.array(joint_rmsd_list)
joint_per_atom_rmsd_arr = np.array(joint_per_atom_list)
prot_in_joint_rmsd_arr = np.array(prot_in_joint_list)
lig_in_joint_rmsd_arr = np.array(lig_in_joint_list)
print(f"Evaluated {len(prot_rmsd_arr)} complexes")
print()

def _print_rmsd_stats(name, arr):
    print(f"{name}:")
    print(f"  Mean:   {arr.mean():.4f}")
    print(f"  Median: {np.median(arr):.4f}")
    print(f"  Std:    {arr.std():.4f}")

_print_rmsd_stats("Protein backbone RMSD – per-atom (Å)", prot_rmsd_arr)
print()
_print_rmsd_stats(
    "Protein backbone RMSD – Kabsch-aligned (Å)", prot_rmsd_aligned_arr,
)
print()
_print_rmsd_stats("Ligand heavy-atom RMSD – per-atom (Å)", lig_rmsd_arr)
print()
_print_rmsd_stats(
    "Ligand heavy-atom RMSD – Kabsch-aligned (Å)", lig_rmsd_aligned_arr,
)

def plot_rmsd_hist(ax, arr, title):
    ax.hist(arr, bins=50, edgecolor="none", alpha=0.8)
    ax.axvline(np.median(arr), color="r", linestyle="--",
               label=f"median={np.median(arr):.3f} Å")
    ax.axvline(arr.mean(), color="orange", linestyle="--",
               label=f"mean={arr.mean():.3f} Å")
    ax.set_xlabel("RMSD (Å)")
    ax.set_ylabel("Count")
    ax.set_title(title)
    ax.legend()

_fig, _axes = plt.subplots(2, 2, figsize=(14, 10))
plot_rmsd_hist(_axes[0, 0], prot_rmsd_arr, "Protein backbone – per-atom")
plot_rmsd_hist(
    _axes[0, 1], prot_rmsd_aligned_arr, "Protein backbone – Kabsch-aligned",
)
plot_rmsd_hist(_axes[1, 0], lig_rmsd_arr, "Ligand heavy-atom – per-atom")
plot_rmsd_hist(
    _axes[1, 1], lig_rmsd_aligned_arr, "Ligand heavy-atom – Kabsch-aligned",
)
_fig.tight_layout()
_fig

```

[illegible]

[illegible]

Evaluated 2000 complexes

Protein backbone RMSD – per-atom (Å):

Mean: 0.9436

Median: 0.9374

Std: 0.1743

Protein backbone RMSD – Kabsch-aligned (Å):

Mean: 0.8986

Median: 0.8923

Std: 0.1709

Ligand heavy-atom RMSD – per-atom (Å):

Mean: 0.3702

Median: 0.3525

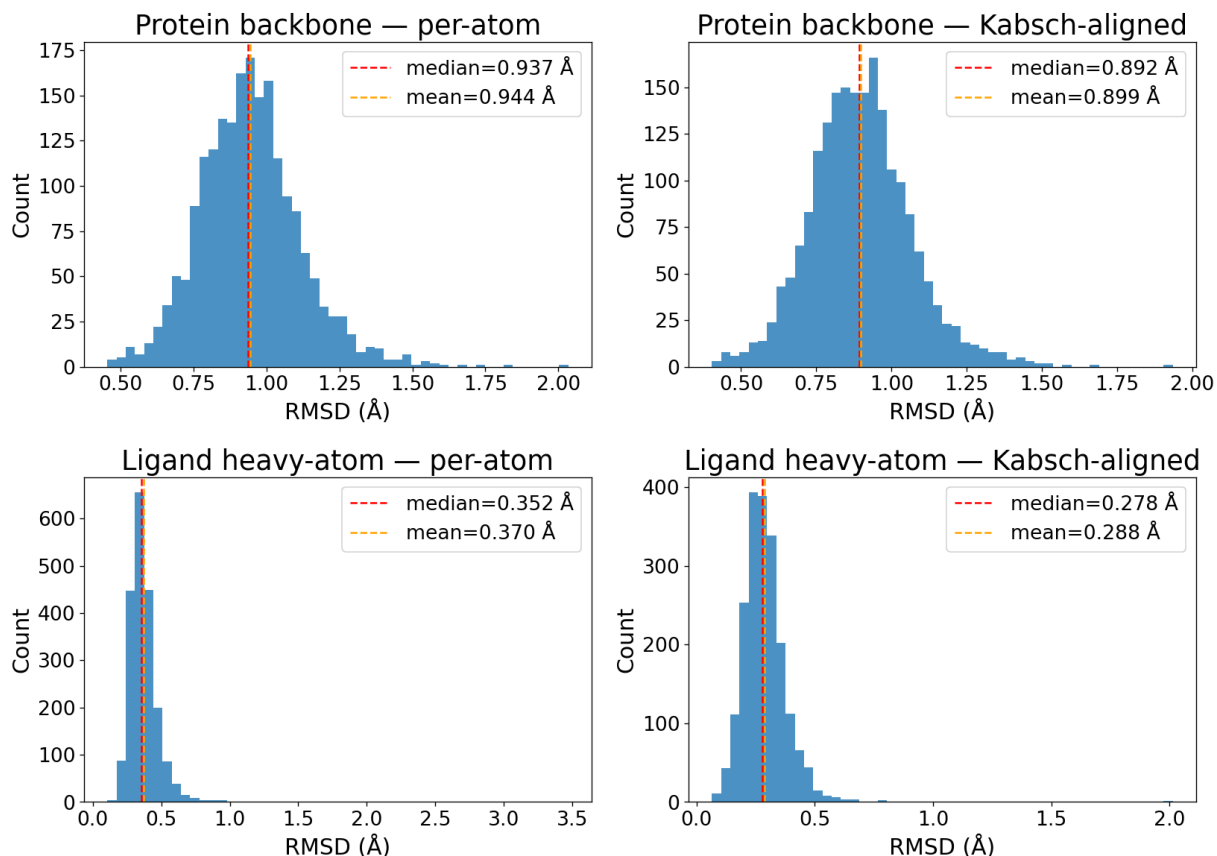
Std: 0.1209

Ligand heavy-atom RMSD – Kabsch-aligned (Å):

Mean: 0.2875

Median: 0.2778

Std: 0.0939



4.1 Whole-complex RMSD

タンパク質バックボーン + リガンド重原子を 1 つの剛体として連結した 複合体レベルの RMSD。

- **per-atom (no alignment):** 両者は同じ pocket canonical frame に 載っている
ので、そのまま全原子で RMSD を取る。生成 pipeline 全体の "絶対座標" 精度。

- **joint Kabsch:** 複合体まるごとを rigid-body アラインしてから RMSD。内部形状（相対ポーズ）の精度。
- **per-component in joint frame:** joint Kabsch 後に protein 部分と ligand 部分それぞれの残差を分解したもの。

```
In [ ]: def _print_joint(name, arr):
    print(f"{name}:")
    print(f"  Mean:   {arr.mean():.4f}")
    print(f"  Median: {np.median(arr):.4f}")
    print(f"  Std:    {arr.std():.4f}")

    _print_joint(
        "Whole-complex RMSD – per-atom (no alignment, Å)",
        joint_per_atom_rmsd_arr,
    )
    print()
    _print_joint("Whole-complex RMSD – joint Kabsch (Å)", joint_rmsd_arr)
    print()
    _print_joint("Protein component in joint frame (Å)", prot_in_joint_rmsd_arr)
    print()
    _print_joint("Ligand component in joint frame (Å)", lig_in_joint_rmsd_arr)

    _fig, _axes = plt.subplots(2, 2, figsize=(18, 12))
    plot_rmsd_hist(
        _axes[0, 0],
        joint_per_atom_rmsd_arr,
        "Whole complex – per-atom (no align)",
    )
    plot_rmsd_hist(_axes[0, 1], joint_rmsd_arr, "Whole complex – joint Kabsch")
    plot_rmsd_hist(_axes[1, 0], prot_in_joint_rmsd_arr, "Protein in joint frame")
    plot_rmsd_hist(_axes[1, 1], lig_in_joint_rmsd_arr, "Ligand in joint frame")
    _fig.tight_layout()
    _fig
```

Whole-complex RMSD – per-atom (no alignment, Å):

Mean: 0.8482
Median: 0.8401
Std: 0.1468

Whole-complex RMSD – joint Kabsch (Å):

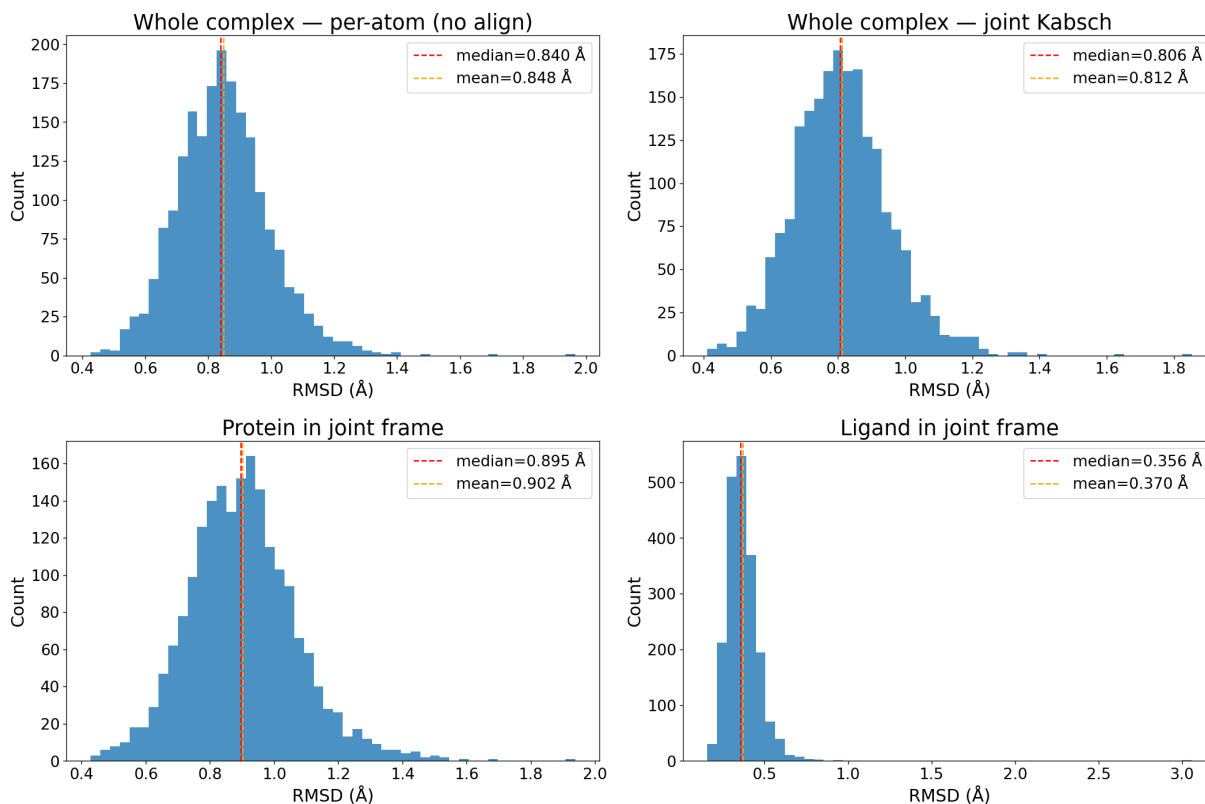
Mean: 0.8118
Median: 0.8059
Std: 0.1438

Protein component in joint frame (Å):

Mean: 0.9023
Median: 0.8954
Std: 0.1693

Ligand component in joint frame (Å):

Mean: 0.3702
Median: 0.3561
Std: 0.1080



5. Latent space (t-SNE)

Codebook ベクトルとエンコーダ出力を 2 次元射影し量子化の質を確認する。

```
In [ ]: from sklearn.manifold import TSNE

def plot_latent_tsne(encoder_z, codebook, name, n_samples=5000):
    z_arr = encoder_z.cpu().numpy()
    cb_arr = codebook.embedding.cpu().detach().numpy()
    rng = np.random.default_rng(42)
    idx = rng.choice(len(z_arr), min(n_samples, len(z_arr)), replace=False)
    z_sub = z_arr[idx]
    combined = np.vstack([z_sub, cb_arr])
    tsne = TSNE(
        n_components=2, random_state=42,
        perplexity=min(30, len(combined) // 4),
    )
    embedded = tsne.fit_transform(combined)
    z_emb = embedded[: len(z_sub)]
    cb_emb = embedded[len(z_sub) :]
    _fig, ax = plt.subplots(figsize=(10, 8))
    ax.scatter(
        z_emb[:, 0], z_emb[:, 1], s=4, alpha=0.3, c="steelblue",
        label="Encoder output",
    )
    ax.scatter(
        cb_emb[:, 0], cb_emb[:, 1], s=40, c="red",
        marker="x", linewidths=1.2, label="Codebook",
    )
```

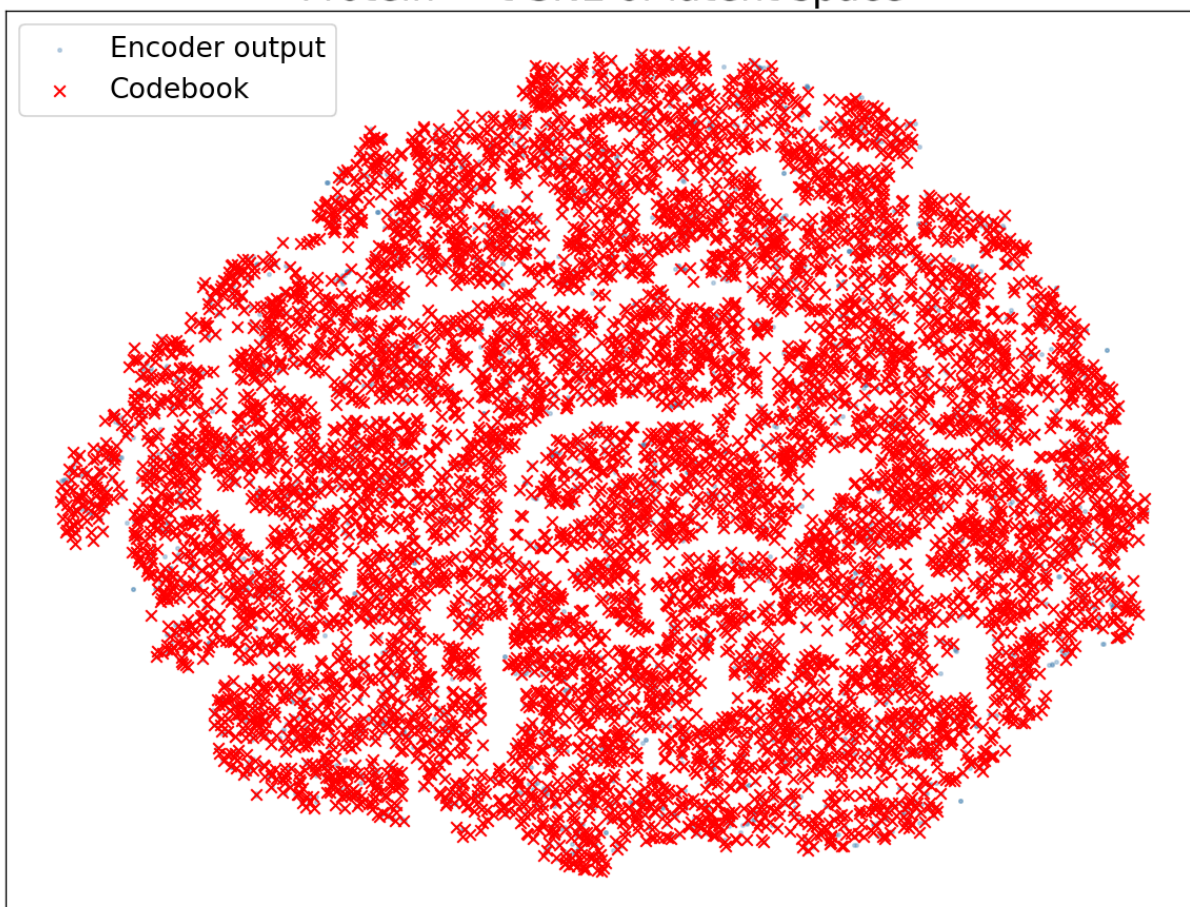
```

ax.set_title(f"{name} — t-SNE of latent space")
ax.legend()
ax.set_xticks([])
ax.set_yticks([])
_fig.tight_layout()
plt.show()

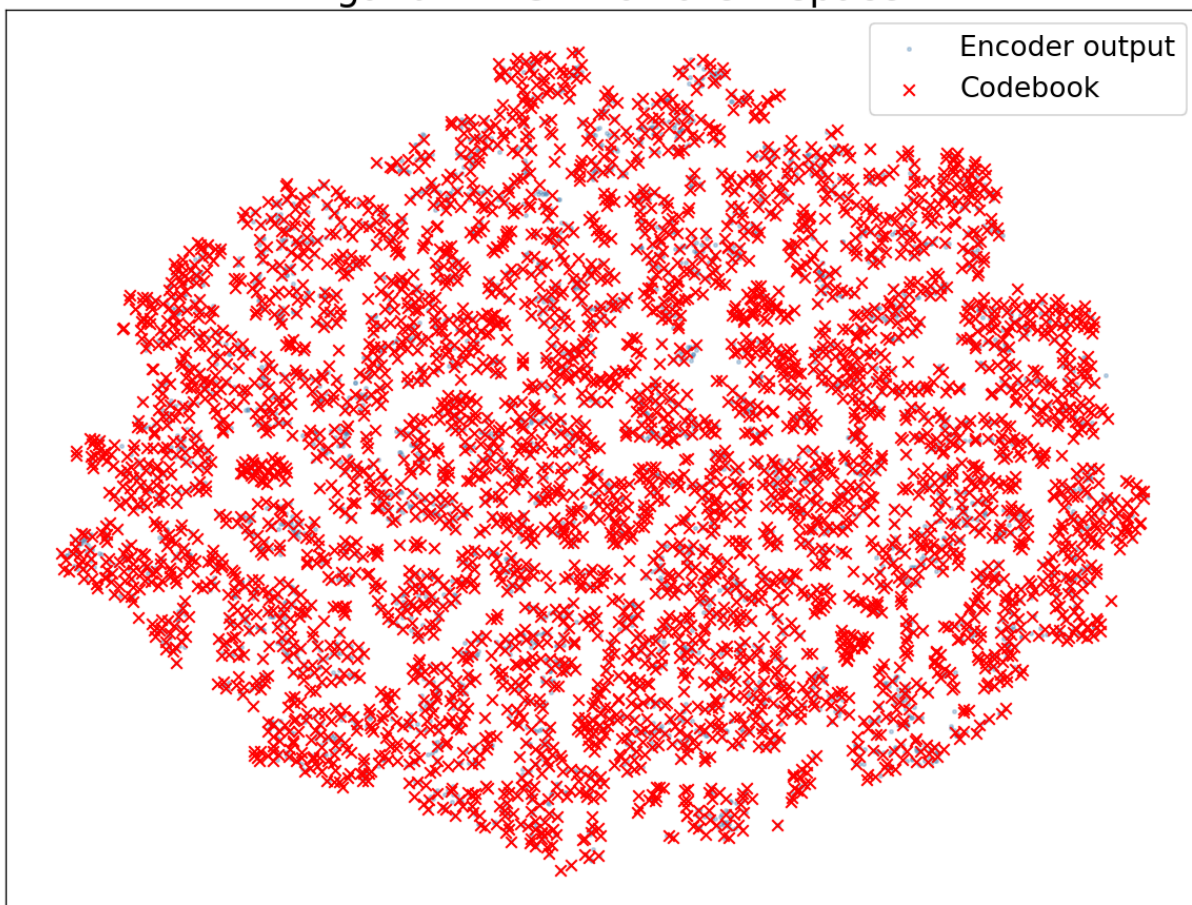
plot_latent_tsne(prot_out["z"], protein_vqvae.codebook, "Protein")
plot_latent_tsne(lig_out["z"], ligand_vqvae.codebook, "Ligand")
_ = config # keep cell deps

```

Protein — t-SNE of latent space



Ligand — t-SNE of latent space



6. Summary table

```
In [ ]: import pandas as pd

prot_idx_np = prot_out["indices"].cpu().numpy()
lig_idx_np = lig_out["indices"].cpu().numpy()
prot_probs = np.bincount(
    prot_idx_np, minlength=config.protein.codebook_size,
).astype(float)
prot_probs = prot_probs / prot_probs.sum()
prot_probs_nz = prot_probs[prot_probs > 0]
prot_ppl = np.exp(-np.sum(prot_probs_nz * np.log(prot_probs_nz)))
lig_probs = np.bincount(
    lig_idx_np, minlength=config.ligand.codebook_size,
).astype(float)
lig_probs = lig_probs / lig_probs.sum()
lig_probs_nz = lig_probs[lig_probs > 0]
lig_ppl = np.exp(-np.sum(lig_probs_nz * np.log(lig_probs_nz)))

summary = pd.DataFrame(
    {
        "Metric": [
            "Codebook size",
            "Latent dim",
            "Coord MSE per-atom ( $\text{\AA}^2$ )",
```

```

        "Coord RMSE per-atom (Å)",
        "Element / AA accuracy",
        "Per-atom RMSD (Å)",
        "Kabsch RMSD (Å)",
        "Active codes",
        "Utilization",
        "Perplexity",
        "Perplexity (normalized)",
    ],
    "Protein": [
        config.protein.codebook_size,
        config.protein.latent_dim,
        f"{prot_metrics['coord_mse']:.4f}",
        f"{prot_metrics['coord_rmse']:.4f}",
        f"{prot_metrics['categorical']['aa']['accuracy']:.4f}",
        f"{prot_rmsd_arr.mean():.4f}",
        f"{prot_rmsd_aligned_arr.mean():.4f}",
        len(np.unique(prot_idx_np)),
        f"{len(np.unique(prot_idx_np)) / config.protein.codebook_size:.4f}",
        f"{prot_ppl:.1f}",
        f"{prot_ppl / config.protein.codebook_size:.4f}",
    ],
    "Ligand": [
        config.ligand.codebook_size,
        config.ligand.latent_dim,
        f"{lig_metrics['coord_mse']:.4f}",
        f"{lig_metrics['coord_rmse']:.4f}",
        f"{lig_metrics['categorical']['element']['accuracy']:.4f}",
        f"{lig_rmsd_arr.mean():.4f}",
        f"{lig_rmsd_aligned_arr.mean():.4f}",
        len(np.unique(lig_idx_np)),
        f"{len(np.unique(lig_idx_np)) / config.ligand.codebook_size:.4f}",
        f"{lig_ppl:.1f}",
        f"{lig_ppl / config.ligand.codebook_size:.4f}",
    ],
}
)
summary

```

	Metric	Protein	Ligand
0	Codebook size	4096	2048
1	Latent dim	16	8
2	Coord MSE per-atom (Å ²)	0.8731	0.1644
3	Coord RMSE per-atom (Å)	0.9344	0.4054
4	Element / AA accuracy	0.9740	0.9984
...	...	...	...
6	Kabsch RMSD (Å)	0.8986	0.2875
7	Active codes	7956	4078
8	Utilization	1.9424	1.9912
9	Perplexity	6253.4	3480.0
10	Perplexity (normalized)	1.5267	1.6992

11 rows × 3 columns